

ICT703 Programming

Module 8 Testing

Observations about Testing

- “Testing is the process of executing a program with the intention of finding errors.” – Myers
- “Testing can show the presence of bugs but never their absence.” – Dijkstra
- “Testing is about raising your credibility that your program works.” – Andrew Lang

Software verification and validation



- **Verification and validation** is intended to show that a system conforms to its **specification** and meets the **requirements** of the system customer
- Involves checking and review processes and system testing.
- System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.

Verification vs. Validation

- Verification:

"Are we building the product right?"

- Tests against a simulation.
- The software should conform to its specification.

- Validation:

"Are we building the right product?"

- Tests against a real world situation.
- The software should do what the user really requires.

Verification & Validation Goals

- Establish **confidence** that the software is fit for purpose.
- Two principal objectives
 - The **discovery of defects** in a system
 - The assessment of whether or not the system is **useful and useable** in an operational situation.
- This ***does not mean:***
 - completely free of defects
 - correct
- Rather, it must be ***good enough for its intended use*** and the type of use will determine the degree of confidence that is needed.

Verification & Validation confidence

- Depends on system's purpose, user expectations and marketing environment
 - **Software function**
 - The level of confidence depends on how critical the software is to an organization.
 - **User expectations**
 - Users may have low expectations of certain kinds of software.
 - **Marketing environment**
 - Getting a product to market early may be more important than finding defects in the program.

Good Testing Practices

- A good test case is one that has a high probability of detecting an undiscovered defect, not one that shows that the program works correctly
- A necessary part of every test case is a description of the expected result
- Avoid nonreproducible or on-the-fly testing
- Write test cases for valid as well as invalid input conditions.
- Thoroughly inspect the results of each test

Good Testing Practices

- As the number of detected defects in a piece of software increases, the probability of the existence of more undetected defects also increases
- Assign your best people to testing
- Ensure that testability is a key objective in your software design
- Never alter the program to make testing easier
- Testing, like almost every other activity, must start with objectives

Testing questions

- Should I test my own code, or should somebody else?
- Which code of my project should I test the most/least?
- Can I test all possible inputs to see whether something works?
- How do I know if I've tested well/enough?
- What constitutes a good or bad test case method?
- Is it good or bad if a test case fails?
- What if a test case itself has a bug in it?

Levels of Testing

- Unit Testing
- Integration Testing
- Validation Testing
 - Regression Testing
 - Alpha Testing
 - Beta Testing
- Acceptance Testing

Unit Testing

- Algorithms and logic
- Data structures (global and local)
- Interfaces
- Independent paths
- Boundary conditions
- Error handling

Workshop Activity:

Given a `Date` module with the following methods:

```
• def Date(year, month, day)           # creates a date for supplied info
• def Date()                           # create date for today
• def getDate(), getMonth(), getYear() # returns an int
• def addDays(days)                    # advances a date by days
• def daysInMonth()                   # returns an int
• def dayOfWeek()                     # returns a string e.g. "Sunday"
• def isLeapYear()                    # returns true or false
• def nextDay()                        # advances by 1 day
```

- Come up with unit tests to check the following:
 - That no `Date` object can ever get into an invalid state.
 - That the `addDays` method works properly.
 - It should be efficient enough to add 1,000,000 days in a call.

Black and white box testing

What is the difference between black- and white-box testing?

- **black-box** (procedural) **test**: Written without knowledge of how the class under test is implemented.
 - focuses on input/output of each component or call
- **white-box** (structural) **test**: Written with knowledge of the implementation of the code under test.
 - focuses on internal states of objects and code
 - focuses on trying to cover all code paths/statements
 - requires internal knowledge of the component to craft input
 - example: knowing that the internal data structure for a spreadsheet uses 256 rows/columns, test with 255 or 257

Black-box testing

- black-box is based on requirements and functionality, not code
- tester may have actually seen the code before ("gray box")
 - but doesn't look at it while constructing the tests
- often done from the end user or OO client's perspective
- emphasis on parameters, inputs/outputs (and their validity)

Types of black-box

- requirements based
- positive/negative
 - checks both good/bad results
- boundary value analysis
- decision tables
- equivalence partitioning
 - group related inputs/outputs
- state-based
 - based on object state diagrams
- compatibility testing
- user documentation testing
- domain testing

Boundary testing

- **boundary value analysis:** Testing conditions on bounds between classes of inputs.
- Why is it useful to test near boundaries?
 - likely source of programmer errors ($<$ vs. $\leq$, etc.)
 - language has many ways to implement boundary checking
 - requirement specs may be fuzzy about behavior on boundaries
 - often uncovers internal hidden limits in code
 - example: array list must resize its internal array when it fills capacity

Boundary example

- Imagine we are testing `daysInMonth(month, year)`
 - What are some conditions and boundary tests for this method?
- Possible answers:
 - check for leap years (every 4th yr, no 100s, yes 400s)
 - try years such as: even 100s, 101s, 4s, 5s
 - try months such as: June, July, Feb, invalid values

Decision tables

Status	Status of spouse	Age > 65? > 65?	Age of spouse	Blind? spouse	Spouse blind?	Standard deduction amount
Single	—	NO	—	NO	—	\$4,750
Single	—	NO	—	YES	—	\$5,750
Single	—	YES	—	NO	—	\$5,750
Single	—	YES	—	YES	—	\$6,750
Married, filing separate return	Claimed standard deduction	NO	—	NO	—	\$7,000
Married, filing separate return	Claimed standard deduction	NO	—	YES	—	\$8,000
Married, filing separate return	Claimed standard deduction	YES	—	NO	—	\$8,000
Married, filing separate return	Claimed standard deduction	YES	—	YES	—	\$9,000
Married, filing separate return	Did not claim standard deduction	—	—	—	—	\$0
Married, filing joint return	—	NO	NO	NO	NO	\$9,500
Married, filing joint return	—	YES	—	NO	NO	\$10,500
Married, filing joint return	—	YES	—	YES	—	\$11,500

Equivalence testing

- **equivalence partitioning:**
 - A black-box test technique to reduce # of required test cases.
 - What is it?
- steps in equivalence testing:
 - identify classes of inputs with same behavior
 - test on at least one member of each equivalence class
 - assume behavior will be same for all members of class
- criteria for selecting equivalence classes:
 - *coverage* : every input is in one class
 - *disjointedness* : no input in more than one class
 - *representation* : if error with 1 member of class, will occur with all

White-box testing

Some kinds of white box testing don't involve unit tests:

- "static testing"
 - code walkthroughs, inspections, code reviews
 - static analysis tools
 - Lint (and variants) JiveLint, JLint, [PMD](#), CheckR, JSLint, `php -l`
 - CheckStyle <http://checkstyle.sourceforge.net/>
 - FindBugs <http://findbugs.sourceforge.net/>
 - code complexity analysis tools
 - PMD, CheckStyle, etc.

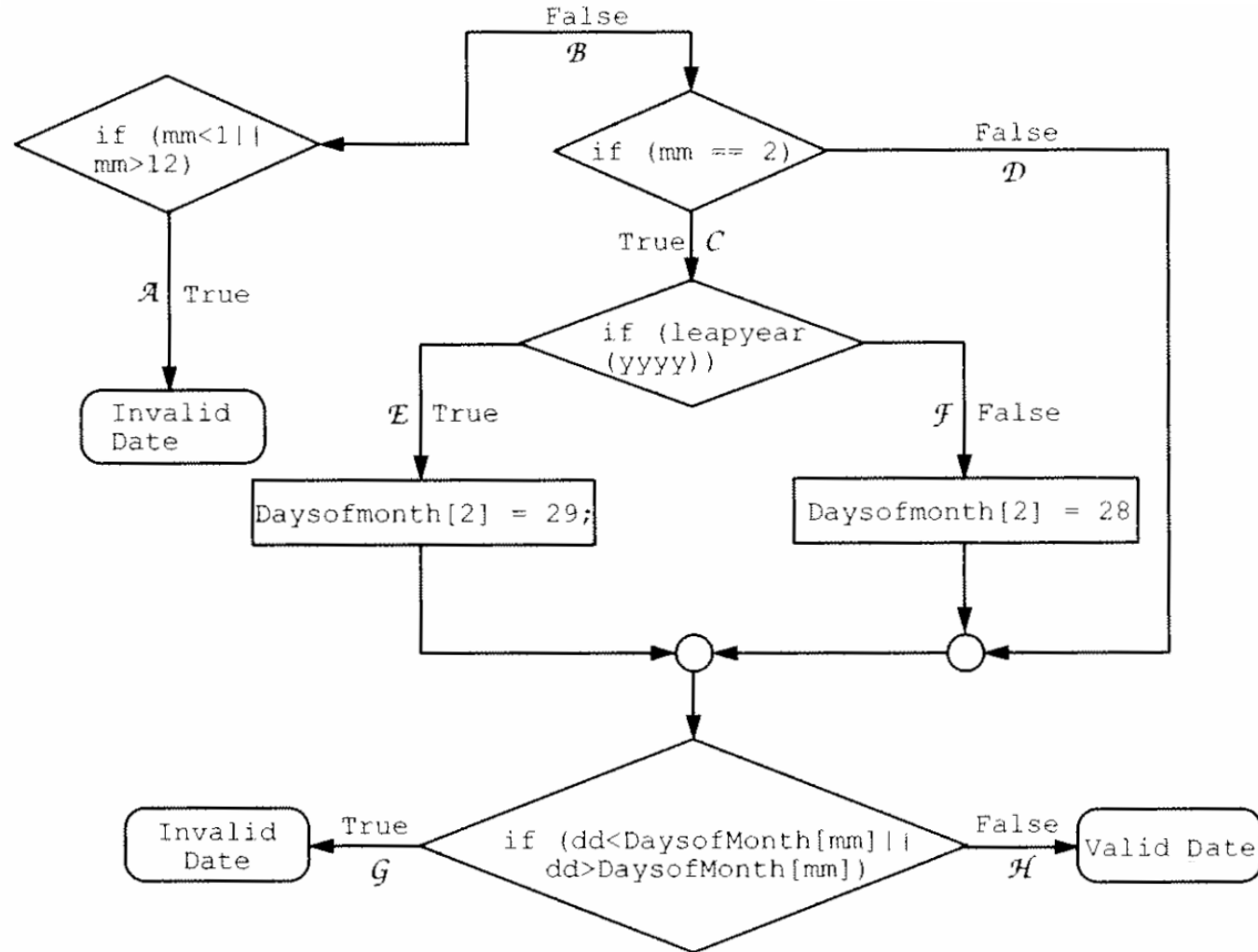
Path testing

- **path testing:** an attempt to use test input that will pass once over each path in the code
 - path testing is *white* box
 - What would be path testing for `daysInMonth(month, year)`?
some ideas:
 - error input: `year < 1`, `month < 1`, `month > 12`
 - one month from `[1, 3, 5, 7, 10, 12]`
 - one month from `[4, 6, 9, 11]`
 - month 2
 - in a leap year, not in a leap year

Code coverage testing

- **code coverage testing:** Examines what fraction of the code under test is reached by existing unit tests.
 - statement coverage - tries to reach every line (impractical)
 - path coverage - follow every distinct branch through code
 - condition coverage - every condition that leads to a branch
 - function coverage - treat every behavior / end goal separately
- Several nice tools exist for checking code coverage

Path coverage example



White box testing is hard

- Developers can't easily spot flaws in their own code.
- Test cases that are too focused on code may not be thinking about how the code
 - is actually going to be used.
- Code coverage tools can give a false sense of security.
 - Just because code is "covered" doesn't mean it is free of bugs.
- Code complexity can be misleading.
 - Complex code is not always bad code.
 - Complexity analysis tools can be overly picky or cumbersome.